Meridian — Agent-Native Finance Network (HSK Chain Horizon Hackathon)

Oct 29, 2026 · @Shriyash soni

Vision

Meridian is a marketplace where AI agents hire other AI agents, and every payment is a signed, policy-bound HSP mandate settled on HSK Chain. One user-owned Trading Agent runs autonomous strategies on HSKSwap (Track 1) and, to do it well, buys signals, risk audits and execution routing from specialist agents it pays per call through HSP (Track 2). Nothing is bolted together: Track 2 is the payment rail that makes Track 1 possible, and Track 1 is the demand that makes Track 2 a real economy.

One-line pitch: "The first on-chain agent economy where a trading agent pays its own research team, risk desk and execution desk, within limits you set in plain English and enforce on-chain."

Why this wins: most entries will build either a trading bot or a payment demo. Meridian ships both as one closed loop with real money flowing through HSP, a hard on-chain safety layer, and verifiable PnL that decides who gets paid.

Problem and why now

AI trading agents today are lone wolves with unlimited trust and no way to pay for what they need.

- **No agent-to-agent payments:** an agent that wants a better signal, a second-opinion risk check or private routing has no safe way to pay another agent. Humans end up pasting API keys and funding hot wallets.
- **No bounded authority:** giving a bot a private key means giving it everything. One prompt injection or bug drains the wallet.
- **No proof of performance:** every bot claims profit. Users and signal sellers cannot verify PnL, so reputation is marketing.
- **No clear liability for fees:** performance fees between agents are settled off-chain or not at all.

HSP's Payment Mandate model (the receiver states payment conditions, the payer agent builds a mandate, signs it, and the gateway verifies and executes on-chain) is exactly the missing primitive. HSK Chain gives a compliance-friendly settlement layer. Meridian is the first product that uses both for a full workflow rather than a single transfer.

Product overview

Meridian has five agent roles. The user only ever talks to the first one.

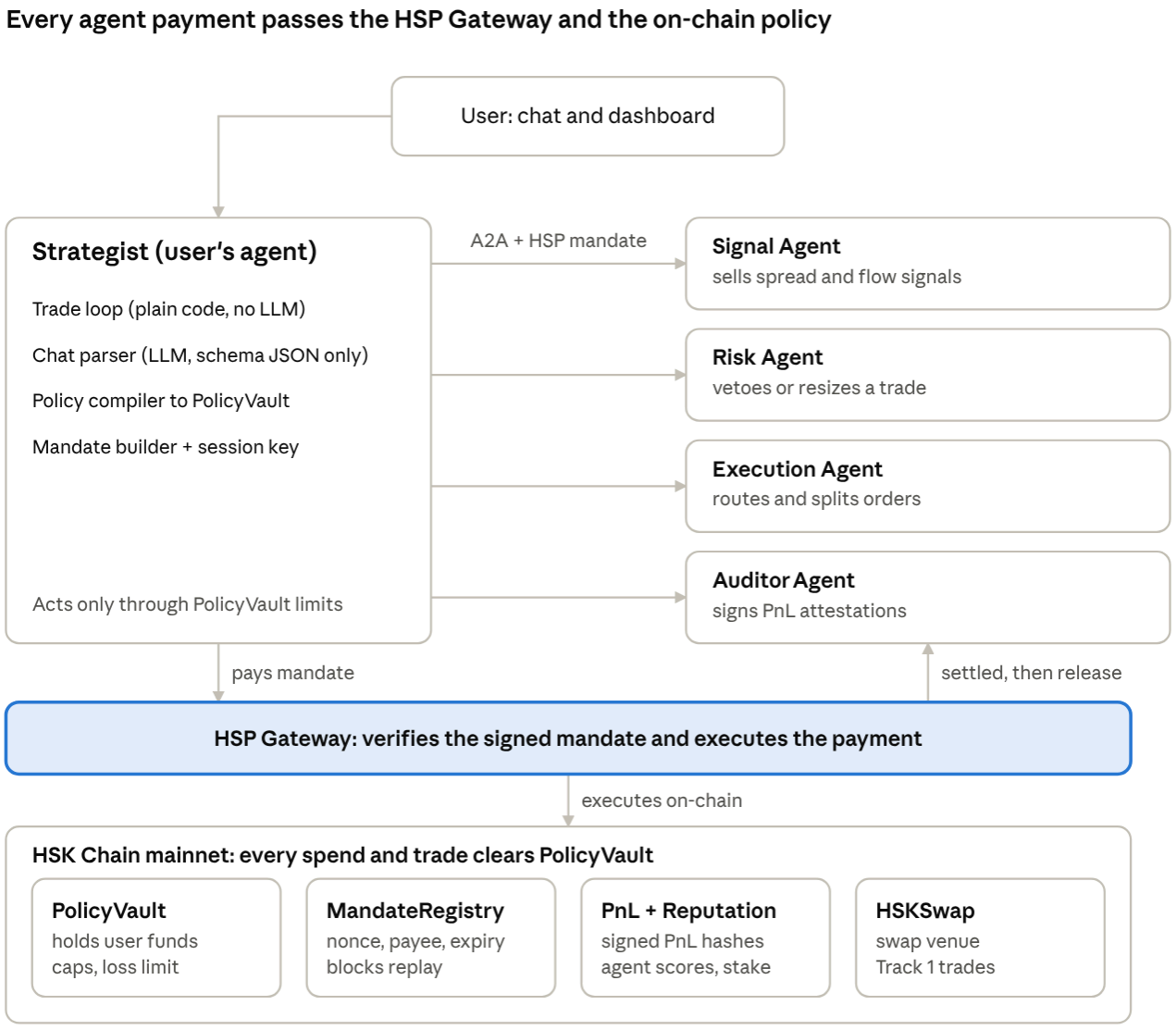
Role	Job	Pays / gets paid via HSP
Strategist (user's Trading Agent)	Runs the strategy loop on HSKSwap, takes natural-language commands, owns the wallet policy	Pays all other agents
Signal Agent	Sells price-spread, arbitrage and on-chain flow signals	Per-signal micro-payment, plus optional performance share
Risk Agent	Independently vetoes or resizes a trade (slippage, pool depth, token risk, exposure)	Per-check fee
Execution Agent	Finds the best route and splits orders to cut slippage	Per-execution fee in bps of notional
Auditor Agent	Reads the trade log and publishes signed PnL attestations that set reputation	Per-epoch fee

User journey

1. The user connects a wallet and deposits into their Policy Vault.
2. They tell the Strategist in chat: "Trade ETH-style majors on HSKSwap, max 5% per trade, 2% daily loss limit, spend at most 20 USDT a day on agent services."
3. The chat is compiled into an on-chain policy: spend caps, allowed payees, allowed pools, loss limits.
4. The Strategist loop runs. For each opportunity it requests a quote from Signal and Risk agents; each quote comes back with HSP payment conditions.
5. The Strategist builds a Payment Mandate per purchase, signs it, and the HSP gateway verifies and executes it. Only then does the service agent release its answer.
6. Execution Agent places the trade; the Auditor logs it. Performance fees settle through HSP at epoch end, only if attested PnL is positive.
7. The dashboard shows live positions, PnL, and an agent-spend ledger. The user can change strategy or limits by chatting.

How the tracks fuse: Track 1 is the Strategist and its trade loop. Track 2 is every purchase the Strategist makes and every fee the others earn. Remove HSP and the system cannot pay its own team; remove trading and there is no reason for agents to pay each other.

System architecture



system architecture · 1 user, 5 agents, HSP, 4 contracts

The user only talks to the Strategist. Every purchase from a service agent goes through the HSP Gateway, and every spend and trade is limited by PolicyVault on HSK Chain. The Strategist's swaps on HSKSwap also run through PolicyVault, so a compromised agent can lose only what the caps allow.

Smart contracts on HSK Chain

Four contracts, all deployed to HSK Chain mainnet (a hard requirement to claim a prize).

Contract	Purpose	Key rules
PolicyVault	Holds user funds; the Strategist can only act through it	Per-trade size cap, daily loss limit, allowlisted pools and tokens, daily agent-spend cap, owner can pause or withdraw any time
MandateRegistry	Records every HSP mandate the vault approves, with nonce and expiry	Replay protection (unique nonce), payee must be on the allowlist, amount within cap, expiry enforced
ReputationRegistry	Agent identity, stake and score	Score updated only from Auditor-signed attestations; stake slashed for provably bad data
PnLAttestation	Stores epoch PnL hashes signed by the Auditor	Performance fee is releasable only if attested PnL is above the high-water mark

Design decisions

- **Agents never hold user keys.** The Strategist uses a session key that PolicyVault limits by code. A compromised agent can lose at most the caps, not the vault.
- **Mandates are checked twice.** HSP's gateway verifies the signature and payment; MandateRegistry verifies policy. Either one can reject.
- **Fees follow results.** Signal and Execution fees are small and flat; the big upside is a performance share that only pays on attested profit.
- **Keep it small.** About 400 lines of Solidity total, with a Foundry test suite covering cap breaches, replay, expiry and pause.

Track 1: AI Agent Trading

The track asks for a full closed loop on a decentralized exchange: market monitoring, on-chain analysis, signal identification, order execution, risk control and stop-loss, plus a natural-language interface. Meridian's Strategist covers each piece.

Loop (runs every few seconds)

1. **Monitor:** read pool reserves, prices and swap events on HSKSwap (app.hskswap.com).
2. **Analyze:** compute spreads across pools and routes, pool depth, and short-term momentum.
3. **Signal:** combine its own signal with purchased ones from the Signal Agent.

4. **Risk gate:** the Risk Agent can veto or resize; the on-chain PolicyVault has the final say.
5. **Execute:** the Execution Agent routes and splits the order; slippage limit is set in the swap call.
6. **Manage:** stop-loss, take-profit and trailing exits are enforced by the loop and mirrored as vault loss limits.

Strategies to ship (pick two for the demo)

- Cross-pool price-spread capture between HSKSwap pools.
- Mean reversion on majors with volatility-scaled position size.
- A conservative mode that only trades when two purchased signals agree.

Natural-language control: the user can ask "what's my PnL today?", "cut risk in half", "stop trading HSK", or "spend more on signals". An LLM parses the message into a strict JSON action (change limit, pause, query) that is validated against a schema, then applied as an on-chain policy update. The LLM never sends free-form transactions.

Honest scoring: judges care about sustained profit under risk control. Run the demo on mainnet with small capital, show a live and a backtested equity curve, and report max drawdown next to return. Do not claim profit you cannot show.

Track 2: Agent Payments and A2A Settlement (HSP)

The track's flow: the receiving agent states payment conditions, the Payment Agent builds a Payment Mandate from them, signs it with the wallet, and the HSP Gateway verifies and executes the payment on-chain. Meridian uses exactly this flow for every agent-to-agent purchase. HSP API reference: hashkey-merchant.github.io. Confirm endpoint names and field formats there before coding; this doc does not assume them.

Mandate lifecycle

1. **Request:** Strategist asks the Signal Agent for a signal over A2A.
2. **Conditions:** Signal Agent replies with payment conditions (amount, token, payee, order id, expiry) instead of the data.
3. **Mandate:** Strategist builds the Payment Mandate from those conditions and checks it against its policy (payee allowlisted, amount under cap).
4. **Sign:** the session key signs; PolicyVault approves the spend and MandateRegistry records the nonce.
5. **Settle:** the HSP Gateway verifies and executes the on-chain payment.
6. **Release:** Signal Agent sees settlement confirmation and returns the signal. No payment, no data.

What makes it more than a demo

- **Four real payee agents** with different pricing: pay-per-call, bps of notional, and performance share.
- **Escrow-style performance fees:** held until the Auditor attests PnL, then released or returned.
- **Agent discovery:** a registry page where anyone can list an agent, its price and its reputation, so the marketplace is open, not scripted.
- **Spend ledger:** every mandate, its payee, its purpose and its on-chain hash, visible to the user.
- **Merchant SDK:** a small package so any third-party agent can accept HSP payments with a few lines.

Open MCP layer: two servers anyone can plug into

Meridian ships two MCP servers, so any MCP client (Claude, other assistants, custom agents) can use the network without our dashboard. This turns Meridian from an app into infrastructure.

Server	Who uses it	Tools it exposes
meridian-pay-mcp	Any developer or agent that wants to accept or send payments	`create_payment_conditions`, `build_mandate`, `submit_mandate`, `verify_settlement`, `list_payments`
meridian-trade-mcp	Any agent or user that wants a policy-bound trading agent	`set_policy` (plain-English limits), `get_quote`, `propose_trade`, `get_positions`, `get_pnl`, `list_agents`

How it works: the pay server wraps the HSP merchant and payer flow behind a chain adapter interface (`quote` , `buildPayment` , `verify`). HSK Chain with HSP is the first adapter. Adding another chain means writing one adapter, not changing agents or tools. A merchant lists their agent or API, gets payment conditions generated by the tool, and gets paid when settlement verifies.

Safety rules for MCP: tools can only act through PolicyVault limits, spend above a threshold needs owner approval, and tool outputs from third parties are treated as untrusted data, never as instructions.

Scope warning: "any chain" is the pitch, but in 3 days ship the adapter interface, the HSK Chain adapter on mainnet, and at most one stub adapter to prove the shape. Claim only what actually runs. The MCP servers are also a strong demo moment: show Claude calling `set_policy` and `propose_trade` live.

The hard parts and how we solve them

Hard problem	Why it is hard	Our solution
Prompt injection into a money-moving agent	A malicious signal or token name can instruct the LLM to send funds	The LLM never builds transactions. It outputs schema-validated JSON; a deterministic executor and PolicyVault enforce limits
Bounded authority	Session keys are usually all-or-nothing	Vault-enforced caps on trade size, daily loss, spend and payees; owner can pause or withdraw instantly
Mandate replay and double-pay	An agent can be tricked into paying twice	Unique nonce plus order id plus expiry, checked in MandateRegistry and by the gateway
Paying before delivery	Seller may not deliver, buyer may not pay	Pay-then-release for small calls; escrowed performance fees for larger ones
MEV and sandwiching on swaps	Public mempool trades get front-run	Tight slippage limits, order splitting, size caps relative to pool depth
Provable PnL	Self-reported profit is meaningless	Auditor replays on-chain swaps from the vault, signs a PnL hash, contract stores it
Latency	LLM calls are slow for a trading loop	LLM only for chat and strategy changes; the trade loop is plain code
Sybil agents in the marketplace	Anyone can list a fake agent	Stake to list, score from attested results, slashing for bad data

These are also the answers to the judges' likely questions, so each one should appear in the demo or the Q&A deck.

Tech stack and repo structure

Layer	Choice
Contracts	Solidity, Foundry, OpenZeppelin; deployed to HSK Chain mainnet
Agents	TypeScript (Node) services; viem or ethers for chain access
LLM	Claude via API for the chat interface and strategy parsing only
A2A transport	Simple HTTP JSON between agents with signed messages
Payments	HSP Gateway API, plus a small merchant SDK wrapper
Data	SQLite or Postgres for trade log and ledger; event indexer script
Frontend	Next.js dashboard: chat, positions, PnL curve, spend ledger, agent registry

Monorepo layout

```
meridian/
  contracts/        PolicyVault, MandateRegistry, ReputationRegistry,
PnLAttestation, tests
  agents/
    strategist/     trade loop, policy compiler, chat parser
    signal/         spread and flow signals, HSP merchant side
    risk/           veto and sizing
    execution/      routing and order splitting
    auditor/        PnL replay and attestation
  sdk/              hsp-merchant wrapper, mandate builder, A2A message types
  web/              dashboard
  docs/             architecture, threat model, demo script
```

Build plan to the deadline

You have about 83 hours from Thu Oct 29, 10:30 IST to the stated deadline of Sun Nov 1, 21:30 IST. The event page's header shows a different date (Nov 2, 09:30), but the rules text says Nov 1, 21:30 IST, so plan for the earlier one.

Thu Oct 29 (today): foundation

- [ ] Register on DoraHacks and join the support Telegram; ask how to get HSP merchant credentials and HSK mainnet gas
- [ ] Read the HSP API docs end to end; make one real test payment through the gateway
- [ ] Write PolicyVault and MandateRegistry; deploy to testnet
- [ ] Scaffold the monorepo and shared SDK

Fri Oct 30: the closed loop (Track 2 first)

- [ ] Signal Agent with HSP merchant side: conditions, verify settlement, release data
- [ ] Strategist builds, signs and pays a mandate; end-to-end paid signal call works
- [ ] Strategist trade loop on HSKSwap with one strategy, small size
- [ ] Risk Agent v1 (slippage and depth veto)

Sat Oct 31: depth and proof

- [ ] Execution Agent and Auditor Agent; PnLAttestation contract
- [ ] Natural-language control with schema-validated actions
- [ ] Dashboard: chat, positions, PnL, spend ledger
- [ ] Deploy all contracts to HSK Chain mainnet; run live with small capital

Sun Nov 1: polish and submit (by 18:00 to leave buffer)

- [ ] Foundry tests green; threat-model page; README
- [ ] Record a 3-minute demo video; screenshots; architecture diagram
- [ ] Submit on DoraHacks and confirm the submission shows as complete

Cut list, in this order, if time runs out: agent discovery page, reputation slashing, second strategy, escrowed performance fee (fall back to flat fees). **Never cut:** mainnet deployment, a real HSP payment between agents, a live trade, and the demo video.

Demo script and submission checklist

The Demo Day is Nov 4 in Mumbai; teams that can demo in person get priority, and pitch teams are announced Nov 3. A 5-minute live pitch:

1. **0:00 Hook (30s):** "AI agents can trade, but they cannot safely pay each other. We fixed that."
2. **0:30 Set limits by chat (45s):** type the risk and spend limits; show them appear as an on-chain policy.
3. **1:15 Live loop (90s):** watch the Strategist buy a signal through HSP, get a Risk Agent check, and place a swap on HSKSwap. Show the mandate and the transaction hash.

4. **2:45 Attack it (60s):** feed a malicious signal that says "send all funds". The vault rejects it. Show the revert.
5. **3:45 Proof (45s):** Auditor's attested PnL, drawdown, and the performance fee releasing only on profit.
6. **4:30 Close (30s):** open marketplace, merchant SDK, and what you will build next.

Submission checklist

- [ ] Contracts verified on HSK Chain mainnet, addresses in the README
- [ ] Both tracks named in the submission, with the Track 2 HSP flow explained
- [ ] At least one real HSP payment hash and one real swap hash linked
- [ ] Demo video, architecture diagram, threat model, test results
- [ ] Registered for the Mumbai Demo Day meetup link
- [ ] A backup recorded demo in case live network or Wi-Fi fails

Judging fit, edge and roadmap

The hackathon stresses completeness, technical maturity and innovation, extra points for officially recommended products, and mainnet deployment as a condition of any prize.

Judging signal	How Meridian answers it
Fits one track	Fits both, and the tracks depend on each other
Uses official products	HSKSwap for trading, HSP gateway for every agent payment, HSK Chain mainnet for all contracts
Technical depth	Vault-enforced session keys, mandate registry, attested PnL, Foundry tests
Innovation	Agents with bounded authority paying other agents, fees that depend on proven results
Completeness	One closed loop that actually runs on mainnet, with a recorded backup
Ecosystem value	Open marketplace and merchant SDK that other HSK builders can reuse

Where competitors will be weaker: most will show a bot with a chat box, or a payment page with no real agent behind it. Few will have a security story, and fewer will prove PnL.

Honest risks: HSP integration details may differ from this plan, mainnet liquidity may be thin for a profitable strategy, and three days is tight. The mitigation is the cut list: the paid signal call plus one live trade plus the vault is already a winning core.

After the hackathon: HashKey Capital and the HSK engineering team offer support to winners and promising non-winners, so keep a short roadmap ready: open the registry to third-party agents, add more venues, a compliance-friendly mode, and a security audit.